

EXPERIMENT 7

Simulate a Collaborative Project using GitHub / GitLab

Mar Baselios College of Engineering and Technology

Department of Computer Science and Engineering | 23CTB40B – Software Engineering Lab

Experiment No.	7
Title	Collaborative Development Simulation using GitHub
CO Addressed	CO4 – Make use of software project management concepts in planning, scheduling, tracking and change management.
Team	3 students – Student A (Team Leader), Student B, Student C

AIM

To simulate a team software project where multiple students collaborate via GitHub using feature branches, pull requests with code review, merge conflict resolution, issue tracking, and a CI/CD pipeline via GitHub Actions.

THEORY

Key Git Concepts

Repository	A directory tracked by Git containing all files and the full commit history.
Commit	A snapshot of the project saved with a unique SHA hash, author, and timestamp.
Branch	A lightweight pointer to a commit. Enables independent parallel development.
Merge	Combines changes from one branch into another. Conflicts occur if both changed the same line.
Pull Request (PR)	GitHub interface to propose merging a feature branch. Enables code review before integration.
Merge Conflict	Occurs when two branches change the same line of a file. Git marks the region with <<<<<<, ===== and >>>>>> and a developer must choose the final content.
GitHub Actions	Built-in CI/CD: runs automated workflows (build, test, deploy) on push or PR events.

Feature Branch Workflow

- `main` – stable, production-ready code. Never commit directly.
- `develop` – integration branch for testing combined features.
- `feature/xxx` – created per feature or issue. Merged via PR after review.
- `hotfix/xxx` – urgent production fixes. Branched from `main`.

STEP-BY-STEP PROCEDURE

Before You Begin (All Members)

- Create a GitHub account at <https://github.com> (use your real name in the profile).
- Install Git from <https://git-scm.com>. On Windows, run every command in **Git Bash**, not Command Prompt.
- Check Python (3.9+) and install PyTest: `python --version` and `pip install pytest pytest-cov`.
- Configure Git once: `git config --global user.name "Your Name"` and `git config --global user.email "your-github-email"`.

- On the first `git push`, sign in through the browser window that opens (Windows). On Linux, enter a Personal Access Token as the password (GitHub → Settings → Developer settings → Personal access tokens → tick "repo" and "workflow").

Step 1 – Create Repository (Team Leader)

Step 67. Log in to <https://github.com>. Click **+** → **New repository**.

Step 68. Name: `lms-collab`. Set to **Public**. Tick **Add a README file**. Click **Create repository**.

► **Take Screenshot 1**

Step 69. Go to **Settings** → **Collaborators** → **Add people**. Invite Students B and C by GitHub username. Students B and C must open the invitation email (or <https://github.com/notifications>) and click **Accept invitation**.

► **Take Screenshot 2**

Step 70. All three members clone the repository:

```
git clone https://github.com/<A-username>/lms-collab.git
cd lms-collab
```

► **Take Screenshot 4**

Step 2 – Create and Assign Issues (Team Leader)

Step 71. Click the **Issues** tab → **New issue**. Create the 3 issues below and use **Assignees** to assign each one:

Issue	Title	Assignee
#1	Implement book search module	Student A
#2	Implement member registration module	Student B
#3	Implement loan management module	Student C

► **Take Screenshot 3**

Step 3 – Create Feature Branches (All Members)

Step 72. Each member runs **only their own** command inside the `lms-collab` folder, then checks with `git branch`:

```
git checkout -b feature/book-search      # Student A only
git checkout -b feature/member-register  # Student B only
git checkout -b feature/loan-management  # Student C only
```

► **Take Screenshot 5**

Step 4 – Implement Feature and Commit (All Members)

Step 73. Each member creates their module and its test file (code given under **SOURCE CODE** below) and runs the tests locally – all 4 tests must PASS. For the merge conflict later, **Students A and B change only line 1 of README.md**: Student A writes `# LMS Collab` – Library Management System (Team <No.>) and Student B writes `# Library Management System – S7 CSE Lab Project`. Student C does not edit README.md.

```
pytest test_book_search.py -v      # Student A
pytest test_member_register.py -v  # Student B
pytest test_loan_management.py -v   # Student C
```

Step 74. Each member commits and pushes their own branch:

```
# Student A
git add book_search.py test_book_search.py README.md
git commit -m "feat: add book search by title and author - closes #1"
git push origin feature/book-search

# Student B
git add member_register.py test_member_register.py README.md
git commit -m "feat: add member registration with validation - closes #2"
git push origin feature/member-register

# Student C
git add loan_management.py test_loan_management.py
git commit -m "feat: add loan due date and fine calculation - closes #3"
git push origin feature/loan-management
```

► Take Screenshot 6

Step 5 – Create Pull Request and Code Review (merge order: A → C → B)

Step 75. Student A goes to GitHub and clicks **Compare & pull request**. Title: Add book search module. Description: Adds search by title and author with 4 test cases. Closes #1. Set **Student B** as Reviewer. Click **Create pull request**.

► Take Screenshot 7

Step 76. Student B (Reviewer) opens the **Files changed** tab, clicks **+** on a line and writes a comment (e.g., "Add a comment describing the BOOKS list"). Then **Review changes** → **Request changes** → **Submit review**.

Step 77. Student A makes the requested change and pushes again. The PR updates automatically – do not open a new PR.

```
git add book_search.py
git commit -m "docs: describe BOOKS sample data as requested in review"
git push origin feature/book-search
```

Step 78. Student B: **Review changes** → **Approve** → **Submit review**. Student A clicks **Merge pull request** → **Confirm merge** → **Delete branch**. Issue #1 closes automatically.

► Take Screenshot 8

► Take Screenshot 9

Step 79. Student C opens a PR for `feature/loan-management` (description ends with Closes #3), reviewer **Student A**. Student A approves; Student C merges and deletes the branch. Then Student B opens a PR for `feature/member-register` (description ends with Closes #2), reviewer **Student C**. GitHub shows **"This branch has conflicts that must be resolved"**, because line 1 of README.md was changed differently on main.

► Take Screenshot 10

Step 6 – Create and Resolve a Merge Conflict (Student B)

Step 80. Bring the latest main into the feature branch. Git reports **CONFLICT (content): Merge conflict in README.md**. (Use `fetch + merge` as shown; `git pull` may stop with a "divergent branches" message.)

```
git checkout feature/member-register
git fetch origin
git merge origin/main
```

Step 81. Open README.md. Git marks the conflict as shown below. Keep the correct final line (or combine both) and **delete all three marker lines** (<<<<<<, =====, >>>>>>).

```
<<<<<< HEAD
# Library Management System - S7 CSE Lab Project
=====
# LMS Collab - Library Management System (Team <No.>)
>>>>>> origin/main
```

Step 82. Commit the resolution and push. The conflict message on the PR disappears. Student C approves; Student B merges and deletes the branch. Issue #2 closes.

```
git add README.md
git commit -m "resolve: merge conflict in README.md"
git push origin feature/member-register
```

► Take Screenshot 11

Step 7 – Set Up GitHub Actions CI/CD Pipeline (Team Leader)

Step 83. Update main and create a branch for the pipeline. Then create the file `ci.yml` inside `.github/workflows/` with the content given under **CI/CD WORKFLOW** below (spaces only, no Tab key).

```
git checkout main
git pull origin main
git checkout -b ci/github-actions
mkdir -p .github/workflows
```

Step 84. Commit and push, then open a PR from `ci/github-actions` into `main` (any teammate as reviewer):

```
git add .github/
git commit -m "ci: add GitHub Actions workflow for automated testing"
git push origin ci/github-actions
```

► Take Screenshot 12

Step 85. Wait for the **green tick** on the PR check, get approval and merge. Open the **Actions** tab → latest run → **test** job → step "4. Run all tests with coverage": it must show **12 passed**. Finally, every member runs the commands below and checks **Issues** → **Closed** shows all 3 issues.

```
git checkout main
git pull origin main
git log --oneline --graph --all
```

► Take Screenshot 13

► Take Screenshot 14

SOURCE CODE – Student Modules and Test Files

Student A – Issue #1

book_search.py

```
# book_search.py (Student A - Issue #1)
BOOKS = [
    {"title": "Software Engineering", "author": "Ian Sommerville"},
    {"title": "Software Engineering: A Practitioner's Approach", "author": "Roger Pressman"},
    {"title": "Clean Code", "author": "Robert C. Martin"},
]

def search_books(keyword, books=BOOKS):
    """Return books whose title or author contains the keyword (case-insensitive)."""
    if not keyword or not keyword.strip():
        raise ValueError("Keyword cannot be empty")
    k = keyword.strip().lower()
    return [b for b in books if k in b["title"].lower() or k in b["author"].lower()]
```

test_book_search.py

```
# test_book_search.py (Student A)
import pytest
from book_search import search_books

def test_search_by_title():
    assert len(search_books("clean")) == 1

def test_search_by_author():
    assert search_books("pressman")[0]["author"] == "Roger Pressman"

def test_search_no_match():
    assert search_books("python") == []

def test_empty_keyword_raises():
    with pytest.raises(ValueError):
        search_books(" ")
```

Student B – Issue #2**member_register.py**

```
# member_register.py (Student B - Issue #2)
class MemberRegistry:
    def __init__(self):
        self.members = {}

    def register(self, member_id, name, email):
        if not name or not name.strip():
            raise ValueError("Name is required")
        if "@" not in email or "." not in email.split("@")[-1]:
            raise ValueError("Invalid email")
        if member_id in self.members:
            raise ValueError("Member ID already exists")
        self.members[member_id] = {"name": name, "email": email}
        return True

    def count(self):
        return len(self.members)
```

test_member_register.py

```
# test_member_register.py (Student B)
import pytest
from member_register import MemberRegistry

@pytest.fixture
def registry():
    return MemberRegistry()

def test_register_valid_member(registry):
    assert registry.register("M01", "Anu", "anu@mbcet.ac.in") is True
    assert registry.count() == 1

def test_duplicate_id_rejected(registry):
    registry.register("M01", "Anu", "anu@mbcet.ac.in")
    with pytest.raises(ValueError):
        registry.register("M01", "Rahul", "rahul@mbcet.ac.in")

def test_invalid_email_rejected(registry):
    with pytest.raises(ValueError):
        registry.register("M02", "Rahul", "rahul.mbcet")
```

```
def test_empty_name_rejected(registry):
    with pytest.raises(ValueError):
        registry.register("M03", " ", "x@y.com")
```

Student C – Issue #3

loan_management.py

```
# loan_management.py (Student C - Issue #3)
from datetime import timedelta

LOAN_DAYS = 14
FINE_PER_DAY = 2 # rupees

def due_date(issue_date):
    return issue_date + timedelta(days=LOAN_DAYS)

def calculate_fine(issue_date, return_date):
    if return_date < issue_date:
        raise ValueError("Return date cannot be before issue date")
    days_late = (return_date - due_date(issue_date)).days
    return max(0, days_late) * FINE_PER_DAY
```

test_loan_management.py

```
# test_loan_management.py (Student C)
from datetime import date
import pytest
from loan_management import due_date, calculate_fine

def test_due_date_is_14_days():
    assert due_date(date(2026, 10, 1)) == date(2026, 10, 15)

def test_no_fine_when_on_time():
    assert calculate_fine(date(2026, 10, 1), date(2026, 10, 15)) == 0

def test_fine_for_3_days_late():
    assert calculate_fine(date(2026, 10, 1), date(2026, 10, 18)) == 6

def test_return_before_issue_raises():
    with pytest.raises(ValueError):
        calculate_fine(date(2026, 10, 5), date(2026, 10, 1))
```

CI/CD WORKFLOW – .github/workflows/ci.yml

```

name: CI - Automated Tests on Push and Pull Request

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: 1. Checkout source code
        uses: actions/checkout@v5

      - name: 2. Set up Python 3.11
        uses: actions/setup-python@v6
        with:
          python-version: '3.11'

      - name: 3. Install test dependencies
        run: |
          pip install pytest pytest-cov

      - name: 4. Run all tests with coverage
        run: |
          pytest -v --cov=. --cov-report=term-missing

```

OBSERVATION TABLE

Activity	Command / GitHub Action	Screenshot Ref.	Done? (Y/N)
Repository created on GitHub	github.com → New repository	Screenshot 1	
Collaborators invited and accepted	Settings → Collaborators → Add people	Screenshot 2	
3 Issues created and assigned	Issues → New issue (×3)	Screenshot 3	
All members cloned the repo	git clone <url>	Screenshot 4	
3 Feature branches created	git checkout -b feature/...	Screenshot 5	
Code and tests committed and pushed by each member	git add → git commit → git push	Screenshot 6	
Pull Request created by Student A	GitHub → Compare & pull request	Screenshot 7	
Code review completed (request changes + approval)	PR → Review changes → Approve	Screenshot 8	
PR merged into main	Merge pull request → Confirm merge	Screenshot 9	
Merge conflict shown on Student B's PR	Two branches editing README.md line 1	Screenshot 10	
Merge conflict resolved	git fetch + git merge + edit + commit	Screenshot 11	
ci.yml pushed and PR opened	git add .github/ → commit → push	Screenshot 12	
GitHub Actions pipeline passed (green tick, 12 tests)	Actions tab → test job passed	Screenshot 13	
All 3 Issues closed after PR merges	Issues tab → Closed	Screenshot 14	

RESULT

A collaborative project simulation was conducted using GitHub. A shared repository was created, 3 issues were tracked, and 3 students contributed via separate feature branches with unit tests. Pull Requests were raised and reviewed, a merge conflict was created and resolved, and a CI/CD pipeline using GitHub Actions was set up to automatically run all 12 tests on every push and PR to the main branch.

RECORD FORMAT — What to write in your record book

Section	Content to Write
Aim	One paragraph.
Theory	Git concepts table. Feature Branch Workflow (bullet points for each branch type).
Step 1 – Repository Setup	Write commands. Paste Screenshots 1, 2 and 4.
Step 2 – Issue Creation	Issues table. Paste Screenshot 3.
Step 3 & 4 – Branches and Commits	Write all git commands and your module + test file. Paste Screenshots 5 and 6.
Step 5 – Pull Request	Write PR and review steps. Paste Screenshots 7, 8, 9, 10.
Step 6 – Merge Conflict	Write the conflict markers and resolution steps. Paste Screenshot 11.
Step 7 – CI/CD	Write the complete YAML file. Paste Screenshots 12 and 13 (green Actions tick).
Observation Table	Fill all 14 rows with Y/N and screenshots attached in sequence.
git log output	Paste or copy the output of: <code>git log --online --graph --all</code>
Result	One result statement.

SUBMISSION – Google Classroom (Every Member)

- **Repository link** (Add → Link): <https://github.com/<A-username>/lms-collab>
- **One PDF** named `Exp7_<RollNo>_<Name>.pdf`: team details (names, roll numbers, GitHub usernames, role A/B/C), Screenshots 1–14 with captions, the `git log --online --graph --all` output, and a 3–4 line note on how the merge conflict was resolved.
- Click **Turn in**. Due: 05/10/2026, 11:59 PM

COMMON ERRORS AND FIXES

Error	Fix
403 / Permission denied on push	Invitation not accepted, or signed in as another user. Accept the invitation; on a shared PC clear the old login in Credential Manager.
"Updates were rejected" on push	Branch is behind. Run <code>git fetch origin</code> and <code>git merge origin/<branch></code> , then push again.
"Need to specify how to reconcile divergent branches"	Use <code>git fetch origin</code> followed by <code>git merge origin/main</code> instead of <code>git pull</code> .
Actions run fails: "no tests ran"	Test files are not on main. Merge all three feature PRs before the <code>ci.yml</code> PR.
ModuleNotFoundError in Actions	File name and import line do not match. Correct the spelling.
Invalid workflow file / workflow not shown	Indentation error (use spaces), or file not at <code>.github/workflows/ci.yml</code> .
Editor (Vim) opens during merge	Type <code>:wq</code> and press Enter.